

Российский университет транспорта РУТ (МИИТ)
Высшая инженерная школа (ВИШ)

Анализ больших текстовых данных и текстовый поиск

Лекция 9

10 ноября 2025

Текстовый поиск

Текстовый поиск — это процесс нахождения в большом множестве текстовых документов тех, которые наиболее соответствуют заданному пользователем запросу по содержанию или смыслу

Он включает:

- анализ запроса и текста документов
- сопоставление терминов и выражений
- оценку релевантности
- ранжирование найденных результатов



Текстовый поиск

Примеры применения:

- Интернет-поиск: Google, Яндекс
- Корпоративные системы: поиск по документации, логам, письмам
- Локальные системы: поиск по библиотекам, чатам, форумам
- Специализированные домены: юридические базы, медицина, научные статьи

Основные типы задач:

- Ключевой поиск (keyword search): точное совпадение слов
- Поиск с ранжированием: упорядочивание по релевантности
- Семантический поиск: учет смысла запроса, синонимов, контекста

Анализ запроса и текста документов

Цель: привести запрос и документы к сопоставимому виду

Основные этапы:

- Токенизация: разбиение текста на слова или токены
- Нормализация: приведение к единому регистру, удаление пунктуации
- Лемматизация / стемминг: сведение слов к базовой форме
- Удаление стоп-слов: исключение частотных слов (“и”, “в”, “на”)
- Обработка опечаток

Сопоставление терминов и выражений

Цель: найти документы, содержащие термины запроса

1. Точное совпадение слов (Boolean search)

Это самый базовый способ. Поисковая система ищет документы, где встречаются слова из запроса — буквально те же самые

Можно управлять логикой с помощью операторов:

- **AND** — оба слова должны быть в документе
- **OR** — достаточно хотя бы одного
- **NOT** — исключить документы, где есть

определённое слово

AND			OR			NOT		
								
INPUT		OUTPUT	INPUT		OUTPUT	INPUT		OUTPUT
A	B	A AND B	A	B	A OR B	A	NOT A	
0	0	0	0	0	0	0	1	
0	1	0	0	1	1	1	0	
1	0	0	1	0	1			
1	1	1	1	1	1			

Сопоставление терминов и выражений

2. Поиск по подстрокам и шаблонам (регулярные выражения)

Здесь поиск выполняется не по целым словам, а по шаблонам — частям слов или их комбинациям. Используются регулярные выражения, которые описывают закономерности в тексте

Пример:

- Шаблон **робот.*** найдёт «робот», «роботы», «роботами»
- Шаблон **обучен.*** найдёт «обучение», «обученный», «обучаются»

Преимущества: гибкость, можно искать по маскам

Недостатки: высокая вычислительная нагрузка и возможные ложные совпадения

Сопоставление терминов и выражений

3. Морфологическое и синонимическое расширение

Реальный язык богат формами и вариантами выражения одной мысли.

Морфологический поиск приводит слова к основной форме (лемме), а синонимическое расширение добавляет близкие по смыслу варианты

Пример:

- Запрос **машина** — поиск также по «автомобиль», «транспорт», «авто»
- Запрос **робототехника** — поиск также по «робот», «роботизированный»

Преимущества: поиск становится умнее и устойчивее к формулировкам

Недостатки: требуется морфологический словарь или модель, иногда возможны лишние совпадения.

Сопоставление терминов и выражений

4. Фразовый поиск

Иногда важна не просто комбинация слов, а их порядок и близость. Фразовый поиск ищет точные выражения — слова подряд, в заданной последовательности

Пример:

- Запрос в кавычках **«искусственный интеллект»** найдёт только те документы, где встречается именно эта фраза, а не просто слова «искусственный» и «интеллект» по отдельности

Преимущество: повышенная точность

Недостаток: может пропустить тексты, где выражение слегка переформулировано

Сопоставление терминов и выражений

5. Поиск по векторным представлениям (семантический поиск)

Современные системы используют векторные модели (Word2Vec, BERT и др.), где каждому слову или предложению соответствует числовой вектор, отражающий его смысл. Поиск выполняется не по совпадению слов, а по сходству смыслов — чем ближе векторы, тем ближе значения

Пример:

- Запрос **купить айфон** найдёт документы с фразами «где приобрести смартфон Apple» или «магазины iPhone». Хотя нет точного совпадения, смысл совпадает

Преимущества: поиск понимает контекст, синонимы и переформулировки

Недостатки: требует обученных моделей и больше вычислений

Оценка релевантности

Цель:

Оценить, насколько найденный документ действительно отвечает смыслу запроса, а не просто содержит нужные слова

Для чего это нужно?

- После сопоставления терминов у нас много совпадений
- Но не все документы одинаково полезны — слово могло встретиться случайно
- Нужен способ измерить степень соответствия между запросом и документом

Оценка релевантности

1. TF-IDF (Term Frequency — Inverse Document Frequency)

Оценивает важность слова:

- TF (частота в документе): чем чаще слово встречается, тем выше его значимость в этом документе
- IDF (редкость в корпусе): чем реже слово встречается в других документах, тем оно информативнее

$$w_{t,d} = tf_{t,d} \times \log \frac{N}{df_t}$$

где $tf_{t,d}$ — частота слова в документе, N — количество документов, df_t — число документов, содержащих это слово

Оценка релевантности

2. BM25 (Best Match 25)

Проблема TF-IDF: он считает, что чем чаще слово встречается в документе, тем важнее этот документ. Но если документ длинный, то там всё встречается чаще, просто потому что слов больше

BM25 — это усовершенствованный вариант TF-IDF, который лучше работает для реальных текстов. Он по-прежнему оценивает, насколько документ соответствует запросу, но делает это более честно: он понимает, что длинные тексты не всегда более релевантны, а частые слова не должны бесконечно увеличивать вес

Оценка релевантности

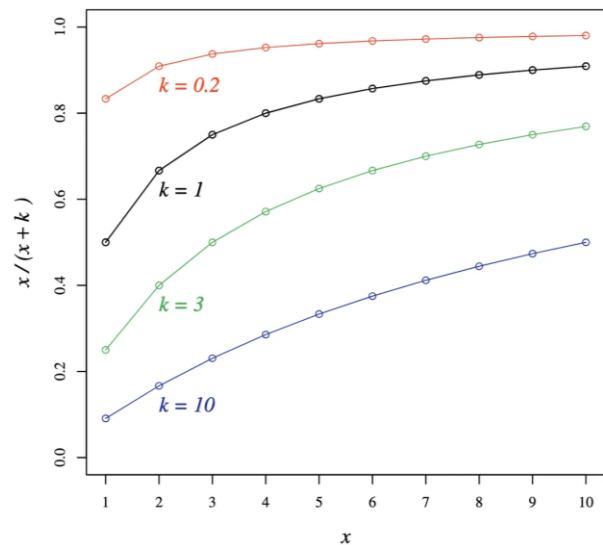
BM25 добавляет к TF-IDF две важные поправки:

- **Учитывает длину документа**

Если документ очень длинный, его вес немного штрафуются. Если документ короткий — наоборот, слегка повышается

- **Сглаживает влияние частоты встречаемости**

TF-IDF считает, что чем чаще слово встречается, тем лучше — без ограничений. BM25 вводит точку насыщения: если слово уже несколько раз встретилось, дальнейшие появления почти не повышают вес



Оценка релевантности

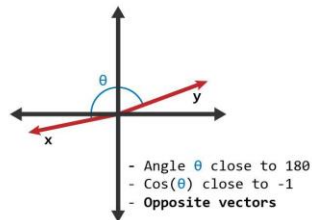
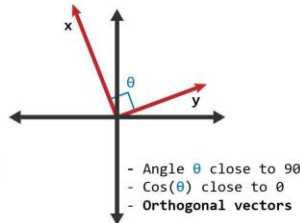
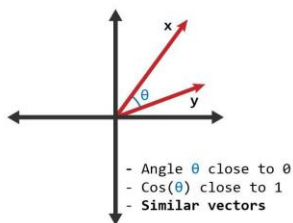
3. Векторные модели (косинусная близость)

Каждое слово получает числовое представление (вес), например, с помощью Word2Vec или GloVe

Документ и запрос превращаются в векторы одинаковой длины

Сравнение идёт по углу между векторами — через косинусную близость.

$$\text{sim}(q, d) = \frac{q \cdot d}{||q|| ||d||}$$



Оценка релевантности

4. Нейронные модели (семантическая релевантность)

Идея:

Перейти от совпадений слов — к пониманию смысла. Модель анализирует не просто что написано, а о чём идёт речь

Современные языковые модели (BERT и др.) преобразуют предложение или документ в эмбединг — плотный вектор, где закодировано значение, контекст и грамматика

Могут считать сходство смыслов между запросом и документом даже без общих слов. Используют те же метрики (косинусная близость), но уже не словарных векторах, а на глубоких контекстных признаках

Ранжирование результатов

Когда система оценила релевантность всех найденных документов, нужно решить, в каком порядке их показать пользователю

Это и есть **ранжирование (ranking)** — упорядочивание документов по убыванию их полезности

Для чего оно нужно?

- Пользователь обычно смотрит только первые результаты (1-2 страницы)
- Система должна поднять вверх документы, которые с наибольшей вероятностью ответят на запрос
- Хорошее ранжирование делает поиск умным: выдача не просто содержит нужные слова, а отражает намерение пользователя

Ранжирование результатов

Алгоритм ранжирования:

Этап 1 — отбор кандидатов (retrieval):

Быстрый поиск по индексу (BM25, TF-IDF и др.) выполняет грубый отбор документов

Этап 2 — ранжирование (ranking):

Для каждого кандидата вычисляется score (оценка релевантности).

Используются дополнительные признаки: длина, клики, свежесть и т.д.

Этап 3 — reranking (переупорядочивание):

Поверх базового ранжирования применяется более сложная модель (например, нейросеть). Она уточняет порядок для топ-N документов.

Ранжирование результатов

Классические методы ранжирования

TF-IDF / BM25. Ранжирование по вычисленному score — чем выше значение, тем выше документ в списке

Эвристическое ранжирование. Добавляются эмпирические факторы: наличие слов из запроса в заголовке, близость слов друг к другу, свежесть публикации, кликабельность или популярность. Ручные правила

Линейная комбинация моделей. Несколько факторов комбинируются с весами:

$$\text{score} = w_1 \cdot \text{BM25} + w_2 \cdot \text{recency} + w_3 \cdot \text{popularity}$$

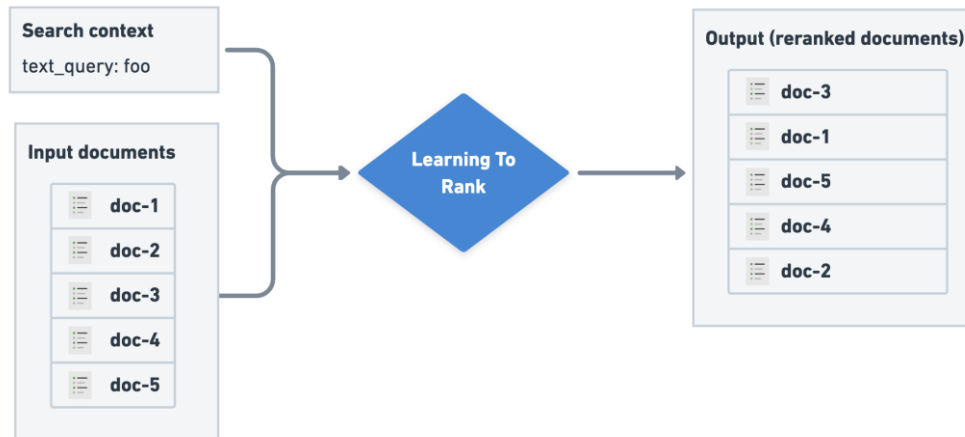
Ранжирование результатов

ML-ранжирование (Learning to Rank)

Когда данных и вычислительных ресурсов достаточно, ранжирование можно обучить

Подходы к ML-ранжированию:

1. Pointwise (точечно)
2. Pairwise (попарно)
3. Listwise (по списку)



Ранжирование результатов

1. Pointwise (точечно)

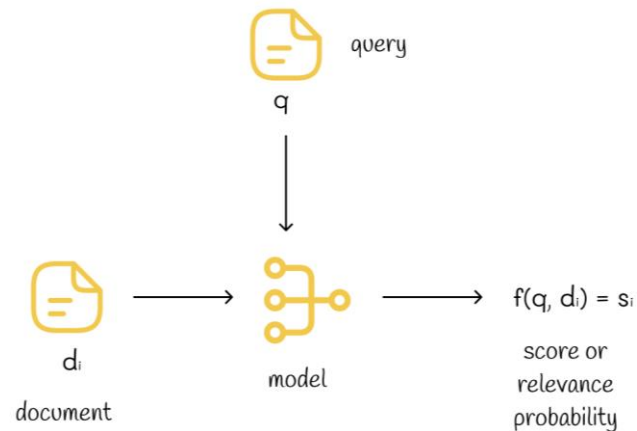
В этом методе задача ранжирования рассматривается как обычная задача классификации. Для каждой пары **запрос–документ** задаётся целевая метка, отражающая, насколько документ релевантен запросу

- 1 — документ релевантен
- 0 — документ нерелевантен

Запрос	Документ	Метка
q_1	d_1	1
q_1	d_2	0
q_1	d_3	1
q_2	d_4	0
q_2	d_5	1



Запрос	Документ	Оценка
q_x	d_{y1}	0.6
q_x	d_{y2}	0.1
q_x	d_{y3}	0.4



Ранжирование результатов

2. Pairwise (попарно)

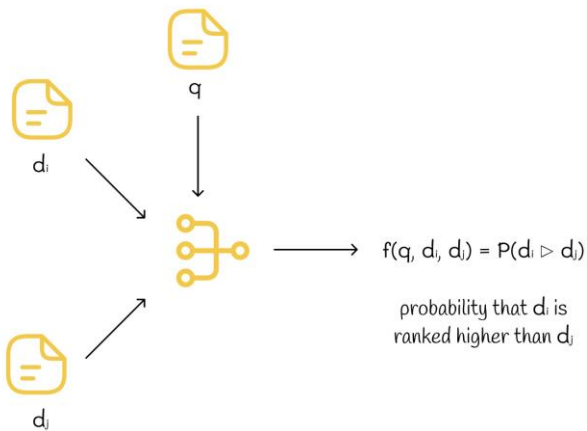
Главный недостаток pointwise-модели в том, что она оценивает каждый документ изолированно, без учёта контекста — то есть без сравнения с другими документами, показанными пользователю. Если пользователь кликнул по документу, это не всегда означает, что он максимально релевантен: возможно, просто остальные результаты были хуже. Если бы система показала другой набор документов, пользовательское поведение могло бы измениться

Чтобы учесть относительную релевантность, используют **pairwise-подход**

Ранжирование результатов

Идея Pairwise

Задача формулируется не как «определи, релевантен ли документ», а как «выбери, какой из двух документов для одного запроса релевантнее». Так модель учится понимать порядок предпочтений между документами, а не их абсолютные оценки



Ранжирование результатов

Алгоритм обучения Pairwise

Запрос	Пара документов	Метка	Интерпретация
q_1	(d_1, d_2)	1	d_1 релевантнее d_2
q_1	(d_2, d_3)	0	d_2 менее релевантен, чем d_3
q_1	(d_1, d_3)	1	d_1 релевантнее d_3
q_2	(d_4, d_5)	0	d_4 менее релевантен, чем d_5

Хотя данные заданы парами, модель обрабатывает каждый документ отдельно:

$$s_1 = f(q_1, d_1), \quad s_2 = f(q_1, d_2), \quad s_3 = f(q_1, d_3)$$

Модель выдаёт оценки релевантности s_i , а затем они сравниваются попарно

Если реальный порядок и предсказанный не совпадают, модель получает штраф

Ранжирование результатов

Особенности и сложности Pairwise подхода:

- Количество пар растёт квадратично с числом документов $O(n^2)$, что делает обучение вычислительно тяжёлым
- Модель оптимизирует локальные сравнения, не формируя глобальный список — из-за этого итоговое ранжирование может содержать противоречия.

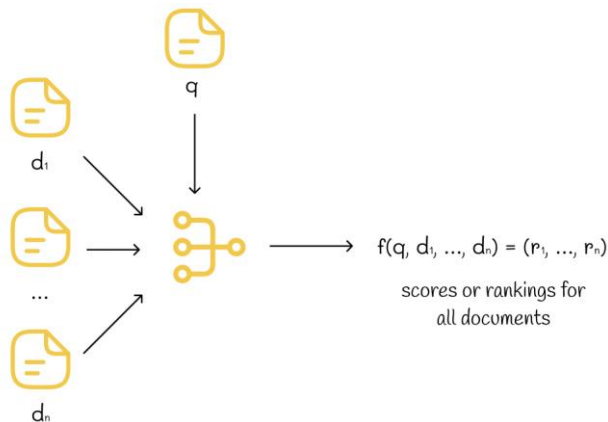
Например:

$$d_1 > d_2, d_2 > d_3, d_3 > d_1$$

Ранжирование результатов

3. Listwise (по списку)

В этом методе система обучается оптимизировать всю последовательность документов, а не отдельные пары или элементы. Главная цель — добиться того, чтобы итоговый порядок документов для каждого запроса максимально соответствовал их реальной релевантности



Ранжирование результатов

Принцип работы Listwise

В отличие от pointwise- и pairwise-подходов, здесь внимание сосредоточено на порядке всей выборки. Модель оценивает не отдельные документы, а всю упорядоченную последовательность — список.

Пример списка для запроса q_1 :

(d_1, d_2, d_3)

Возможные перестановки:

$(d_1, d_2, d_3), (d_1, d_3, d_2), (d_2, d_1, d_3), (d_2, d_3, d_1), (d_3, d_1, d_2), (d_3, d_2, d_1)$

Ранжирование результатов

Этапы обучения Listwise

1) Предсказание оценок. Модель вычисляет значение релевантности для каждого документа:

$$s_1 = f(q_1, d_1), s_2 = f(q_1, d_2), s_3 = f(q_1, d_3)$$

После этого документы сортируются по предсказанным оценкам

2) Правильный порядок. По реальной релевантности формируется эталонный список (например, $d_1 > d_3 > d_2$)

3) Вычисление метрики nDCG (Normalized Discounted Cumulative Gain). nDCG показывает, насколько хорошо предсказанный порядок совпадает с идеальным. Чем выше релевантные документы находятся в списке, тем выше значение nDCG.

Ранжирование результатов

4) Оценка ошибки. Если модель выдала неверный порядок, значение $nDCG$ снижается.

5) Обновление модели. Модель вычисляет градиенты — насколько изменится $nDCG$ при перестановке документов — и корректирует параметры, чтобы улучшить общий порядок

Итог:

Listwise-подход обучает модель оптимизировать всю ранжированную выдачу, а не отдельные элементы, что делает результаты более согласованными и приближенными к пользовательским ожиданиям

Ранжирование результатов

В современных системах итоговое ранжирование выполняется в два этапа

1. Dense Retrieval

Идея: быстрое извлечение кандидатов (retrieval)

- Запрос и документы кодируются отдельно в векторы (эмбеддинги)
- Затем используется поиск ближайших векторов (через косинусную близость)
- Получаем топ-N документов, которые примерно по смыслу похожи

Преимущество — быстро: не нужно прогонять все документы через BERT, достаточно сравнивать векторы

Недостаток — точность ниже, потому что модель не видит реальный контекст пары (запрос + документ)

Ранжирование результатов

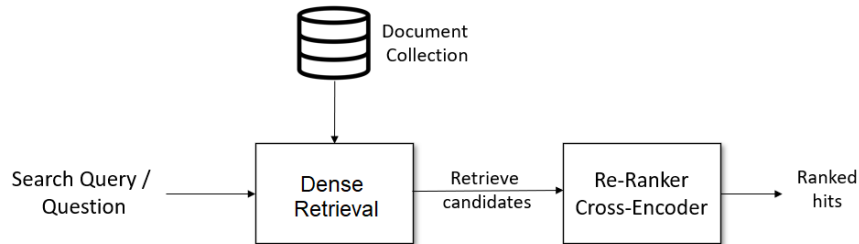
2. Cross-Encoder Reranking

Идея: точное доупорядочивание (reranking)

- После первого этапа (Dense Retrieval) у нас есть N кандидатов
- Теперь каждая пара (запрос, документ) подаётся вместе в BERT
- Модель читает оба текста целиком и предсказывает оценку релевантности

Преимущество — очень точный контекстный анализ

Недостаток — медленно: прогон модели для каждой пары



Метрики качества поиска

Поисковая система должна не просто находить документы, а выдавать лучшие из них первыми. Метрики позволяют измерить, насколько выдача совпадает с ожиданиями пользователя

1. Precision, Recall и F1

Когда мы считаем Precision и Recall, мы не учитываем порядок документов, только сам факт — документ найден или нет

Пример:

Позиция	Документ	Реально релевантен?	Что это для метрики
1	D1	да	TP
2	D2	да	TP
3	D3	да	TP
4	D4	нет	FP
5	D5	нет	FP

Здесь неважно, что, например, D3 мог быть чуть выше — всё равно True Positive

Метрики качества поиска

2. MAP (Mean Average Precision)

Используется, если нужно оценить весь список ранжирования, а не только топ-k

Average Precision (AP):

Для одного запроса берём список результатов и вычисляем среднюю точность в тех позициях, где встречаются релевантные документы

$$AP = \frac{1}{R} \sum_{k=1}^N P(k) \cdot rel(k)$$

- $P(k)$ — precision (точность) на позиции k ,
- $rel(k)$ — 1, если документ на позиции k релевантен, иначе 0
- R — общее количество релевантных документов для данного запроса
- N — длина списка результатов

Метрики качества поиска

$$AP = \frac{1}{R} \sum_{k=1}^N P(k) \cdot rel(k)$$

Пример расчета AP

Позиция	Релевантен?	Precision до этой позиции
1	Да	1/1 = 1.00
2	Нет	1/2 = 0.50
3	Да	2/3 ≈ 0.67
4	Да	3/4 = 0.75
5	Нет	3/5 = 0.60

Общее число релевантных документов в коллекции **R=3**

Теперь берём precision только в тех позициях, где **rel(k)=1**, и считаем среднее:

$$AP = \frac{1}{3} \times (1.0 + 0.67 + 0.75) = 0.81$$

Метрики качества поиска

MAP (Mean Average Precision)

Среднее **AP** по всем запросам:

$$MAP = \frac{1}{Q} \sum_{i=1}^Q AP_i$$

- **Mean Average Precision** показывает, насколько высоко в выдаче появляются релевантные документы
- Если нужные документы стабильно оказываются вверху — MAP высокий

Метрики качества поиска

3. nDCG (Normalized Discounted Cumulative Gain, «Нормализованная взвешенная накопленная выгода»)

Оценивает, насколько полезна выдача для пользователя, при условии, что первые позиции важнее, чем нижние

Почему нужна эта метрика?

Реальный пользователь не просматривает все документы — обычно максимум первую страницу. Поэтому документ, стоящий на 1-м месте, должен весить намного больше, чем документ на 10-м, даже если оба релевантны

Метрики качества поиска

1) Считаем DCG (Discounted Cumulative Gain)

DCG — накопленная польза от документов, где вклад каждого документа уменьшается с позицией в списке

$$DCG = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)}$$

Релевантность может быть разной степени. Например:

- 0 — не релевантен
- 1 — частично полезен
- 2 — полностью по теме
- 3 — отличный, идеальный ответ

Метрики качества поиска

Пример расчета DCG

Допустим, поисковая система выдала 5 документов, а релевантность мы оцениваем по шкале от 0 до 3:

Позиция (i)	Релевантность (rel _i)
1	3
2	2
3	3
4	0
5	1

Расчёт пошагово:

i	rel _i	$2^{rel_i} - 1$	$\log_2(i+1)$	Результат
1	3	7	$\log_2(2)=1.00$	7.000
2	2	3	$\log_2(3)=1.585$	1.892
3	3	7	$\log_2(4)=2.000$	3.500
4	0	0	$\log_2(5)=2.322$	0.000
5	1	1	$\log_2(6)=2.585$	0.387

$$DCG = \sum_{i=1}^k \frac{2^{rel_i} - 1}{\log_2(i + 1)}$$

$$DCG = 7.000 + 1.892 + 3.500 + 0.000 + 0.387$$

$$DCG = 12.779$$

Метрики качества поиска

2) Считаем nDCG (Normalized Discounted Cumulative Gain)

nDCG показывает, насколько полученное ранжирование близко к идеальному

$$nDCG = \frac{DCG}{IDCG}$$

- DCG — Discounted Cumulative Gain для первых k документов в реальной выдаче
- IDCG — тот же показатель, но для идеальной (отсортированной по убыванию релевантности) выдачи
- nDCG всегда лежит в диапазоне от 0 до 1

Метрики качества поиска

Пример расчета nDCG

Для того же примера составим идеальный порядок релевантностей.

Отсортируем по убыванию: [3, 3, 2, 1, 0]

Теперь считаем IDCG по той же формуле:

i	rel _i	2 ^{rel_i} - 1	log ₂ (i+1)	Результат
1	3	7	1.000	7.000
2	3	7	1.585	4.417
3	2	3	2.000	1.500
4	1	1	2.322	0.431
5	0	0	2.585	0.000

$$IDCG = 7.000 + 4.417 + 1.500 + 0.431 + 0.000$$

$$IDCG = 13.348$$

Теперь считаем nDCG:

$$nDCG = \frac{DCG}{IDCG} = \frac{12.779}{13.348} = 0.957$$

Позиция (i)	Релевантность (rel _i)
1	3
2	2
3	3
4	0
5	1

Разметка данных

Откуда берутся оценки релевантности?

Чтобы посчитать любую метрику (Precision, MAP, nDCG), нужен эталон — правильная разметка, где известно, какие документы релевантны и насколько.

Есть два варианта:

1) Ручная разметка (Human Judgments)

Так делают в реальных поисковиках (Google, Яндекс, Bing, Baidu): люди-оценщики получают запрос и набор документов и проставляют каждому оценку по шкале — насколько документ отвечает на запрос

- Разметчик указывает тип запроса, намерение пользователя, оценку релевантности и другие параметры
- Примеры извлекаются из реальных логов поиска

Разметка данных

2) Автоматическая разметка (по логам пользователей)

Можно использовать поведенческие данные: клики, время чтения, возврат на страницу поиска.

Например:

- если пользователь кликнул и читал документ долго — считаем, что релевантен
- если быстро вернулся назад — скорее всего, не подходит

Так создают псевдоразметку (pseudo-labels) для обучения и теста